

TP 0 - Prise en main de PostgreSQL

Le système de gestion de bases de données que nous allons utiliser est **PostgreSQL**. Il a été installé sur la machine `pedago.univ-avignon.fr`.

⚠ Attention : Le serveur `pedago` est exclusivement utilisé comme serveur de bases de données et n'est accessible que par SSH (donc ni par `rlogin` ni par `telnet`).

1°) Accéder à `pedago`

Se connecter depuis le campus :

Taper la commande `ssh uapvXXXXXXXX@pedago.univ-avignon.fr` dans une console en remplaçant les X par votre identifiant `uapv`.

Se connecter depuis l'extérieur :

Si vous n'êtes pas sur le campus, il faut d'abord que vous accédiez au réseau de l'université car le serveur `pedago` n'accepte que des connexions provenant de son propre réseau.

Pour cela, il faut que vous lanciez un VPN. Plus d'informations sur cet accès ici :

<https://dosi.univ-avignon.fr/faq/acces-au-reseau-depuis-l-exterieur-vpn>

2°) Se connecter au SGBD PostgreSQL

Une fois connecté à la machine serveur, il faut se connecter à la base de données, nommé `etd`. Pour cela tapez la commande suivante :

```
uapvXXXXXXXX@pedago01c:~$ psql etd
```

La connexion est établie une fois que le prompt affiche `etd=>` .

Vous êtes directement connecté à votre compte personnel (votre schéma) PostgreSQL.

3°) Nommage des objets (tables, attributs, vues, séquences,...)

Une fois connecté au SGBD, chaque utilisateur est dans son propre « espace » appelé schéma.

Comme dans un système de gestion de fichiers, chaque utilisateur a tous les droits sur ses propres objets de la base et peut les déléguer à d'autres utilisateurs avec

l'instruction **GRANT**. Contrairement à un système de fichiers, les objets des utilisateurs ne sont adressés en fonction de leur emplacement (path) mais en fonction de leur nommage. Dans un SGBD, les objets sont préfixés en fonction de leur appartenance ainsi pour indiquer un attribut particulier d'une table on devrait utiliser le format suivant :

`nom_de_la_base_de_donnée.user_schéma.table.attribut.`

Heureusement, le préfixage n'est obligatoire que lorsqu'il y a ambiguïté, ou que l'on ne veut pas parler de la valeur par défaut, à savoir ; un objet dans la même base, dans le même schéma que celui auquel on est connecté, dans la même table que celle utilisée par la requête.

4°) Quelques commandes, rappels et autres remarques

Point d'attention :

- Une instruction SQL se **termine toujours par un point virgule**.
 - Une commande peut être écrite sur plusieurs lignes. Tant que le point virgule n'est pas tapé, PostgreSQL donne une ligne supplémentaire pour continuer d'écrire la commande. Cela donne lieu à un changement de caractère dans le prompt : le `=` devient un `-`. Pour sortir de cette boucle infernale 2 solutions :
 - Taper un `;` avant de valider la commande par un retour chariot. La commande, composée de l'ensemble des lignes utilisées, est alors exécutée
 - Sinon pour une NON-exécution de la commande, taper `Ctrl+C`.
 - Environnement de travail idéal : il est conseillé d'utiliser un éditeur de texte, graphique si vous le souhaitez, extérieur (e.g. : nedit par exemple). Dans cet éditeur, vous tapez toutes les commandes que vous voulez pour répondre aux questions du TP. Pour les exécuter, il suffit alors de les sélectionner avec le bouton gauche de la souris puis de les copier/coller dans la fenêtre PostgreSQL avec la souris.
- 💡 Tips : Utilisez `Ctrl+Shift+V` pour coller votre commande directement dans le terminal

Commandes PostgreSQL à connaître :

`etd=> \i toto.sql` Lance l'exécution des commandes SQL contenues dans le fichier `toto.sql` (qui doit être situé dans le répertoire à partir duquel vous vous êtes connecté au SGBD, sinon il faut donner le path du fichier, relatif ou absolu)

`etd=> \! commande_unix` Exécute la commande unix sans sortir de PostgreSQL ('ls' par

exemple pour voir si le fichier toto.sql est bien où il doit être)

`etd => \h mot_cle' _SQL` Donne des explications sur le mot clé spécifié

`etd => \set` Affiche les variables d'environnement

`etd => \dt` Liste le nom des tables contenues dans le schéma

`etd => \d nom_table` Liste le nom des attributs composant la table nom_table

`etd => \o toto.log` Spécifie la sortie standard. C'est-à-dire que l'ensemble des résultats est écrit non plus à l'écran mais dans le fichier spécifié. Ne pas abuser des traces, les disques alloués sont vite remplis et cela risquerait de bloquer votre session. Lors de l'utilisation de cette commande, plus aucun résultat n'est affiché sur l'écran. Pour rétablir l'affichage sur l'écran, taper `\o` sans préciser de nom de fichier.

`etd => \?` Affiche toutes les commandes PostgreSQL.

`etd => \q` Sortir de l'environnement PostgreSQL

5°) Écriture de requêtes SQL

Création des tables

Pour le moment votre schéma ne contient aucune table. Vous allez donc créer vos premières tables.

1. Récupérer le script emp.sql sur la plateforme de cours en ligne.
2. Déplacer le fichier téléchargé dans vos données itinérantes
3. Exécutez la command `\i l'endroit/ou/vous/avez/rangé/le/fichier/emp.sql`

Votre première requête SQL

Écrivez la requête SQL qui permet de donner tous les noms des employés puis exécutez là

Vous devriez avoir comme résultat les lignes suivantes :

Noms des employés

SMITH

ALLEN

WARD

JONES

MARTIN

BLAKE

CLARK

SCOTT

KING

TURNER

ADAMS

JAMES

FORD

MILLER